

# Fundamentals of Web Applications

## Backend Development: REST APIs with Express

Lectures 21 and 22 – May 21 and 27, 2026

Michael Soprano – [michael.soprano@uniud.it](mailto:michael.soprano@uniud.it)

University of Udine – Department of Mathematics,  
Computer Science, and Physics (DMIF)

# Outline

1. APIs and RESTful Resources
2. Reading Resources with GET
3. Request Data in Express
4. Creating Resources with POST
5. Validation and Resource Changes

# Organizing Backend Routes

- An Express backend can expose several **routes**
- As routes grow, naming and behavior need a consistent **structure**
- An **Application Programming Interface (API)** turns backend routes into a predictable interface for **data** and **actions**
- REST provides one common style for organizing that interface

# API as a Contract

- An **API** defines an agreement between frontend and backend
- The contract says which HTTP **methods**, **paths**, and request data the frontend can use
- It also defines which **responses** the backend should return
- Stable API rules let the frontend rely on predictable **behavior**

# API Endpoints

- An **endpoint** is one access point exposed by an API
- It combines an HTTP **method** with a **path**
- The endpoint defines which request the client can send
- The backend implements the endpoint with an Express **route**

| Endpoint     | Meaning                         |
|--------------|---------------------------------|
| GET /tasks   | Read the task collection        |
| POST /tasks  | Create a task in the collection |
| GET /tasks/1 | Read one specific task          |

# REST: Representational State Transfer

- **Representational State Transfer (REST)** is a style for organizing API endpoints
- It was introduced to describe design principles behind the **Web**
- REST organizes interaction around **resources** and their representations
- Standard HTTP methods describe how clients read or change those resources

# Resources as API Targets

- REST-style APIs are organized around **resources**
- A resource is something the application exposes or manages
- Examples include **tasks, users, products, comments, or reservations**
- Endpoints describe how clients access and manipulate those resources

# Resource Representations

- The server does not expose its internal resource directly
- It sends a **representation** of that resource
- Representations are often sent as **JSON**
- The frontend reads the representation and updates the **interface**

# Example Resource: Task

- A task is a simple resource with clear **properties**
- It has an **identifier**, a title, and a completion state
- It belongs to a **collection** of tasks

```
{  
  "id": 1,  
  "title": "Review backend slides",  
  "completed": false  
}
```

# Collections and Items

- A **collection** groups resources of the same kind
- `/tasks` can represent the collection of **all tasks**
- `/tasks/1` can represent **one specific task**
- The same naming pattern supports broad and specific **requests**

# Method and Resource

- The **path** answers which resource the request targets
- The HTTP **method** answers what the client wants to do
- The same path can support different operations
- Express implements each method-path pair as a separate **route**

```
GET /tasks → read the collection  
POST /tasks → create inside the collection
```

# API Endpoint Map

- A REST API can be summarized as a small **map of endpoints**
- Each endpoint combines an HTTP **method**, a **path**, and an expected outcome
- The map helps frontend and backend code follow the same **contract**
- Express routes implement the endpoints described by the map

| Method | Path       | Purpose               |
|--------|------------|-----------------------|
| GET    | /tasks     | Read all tasks        |
| GET    | /tasks/:id | Read one task         |
| POST   | /tasks     | Create a new task     |
| PUT    | /tasks/:id | Replace one task      |
| PATCH  | /tasks/:id | Change part of a task |
| DELETE | /tasks/:id | Remove a task         |

# Real API Documentation

- **Readersourcing 2.0** exposes a REST API developed for a real research application
- Its API documentation lists available **methods**, **paths**, request data, and responses
- The documentation can be read before looking at the backend **implementation**
- The same contract can then be implemented as Express **routes**

## Retrieving Representations

- Reading resources means retrieving **representations** from the backend
- A `GET` request should not change the server-side **resource**
- Collection endpoints usually return JSON **arrays**
- Item endpoints usually return one JSON **object** or `404 Not Found`

| Request                     | Target          | Response Shape       |
|-----------------------------|-----------------|----------------------|
| <code>GET /tasks</code>     | Task collection | Array of tasks       |
| <code>GET /tasks/:id</code> | One task        | Task object or error |

# In-Memory Resources

- A first implementation can store resources in an **array**
- Each object represents one task **resource**
- The data exists only while the server process is running
- Persistent data storage requires a database

```
const tasks = [  
  { id: 1, title: "Review backend slides", completed: false },  
  { id: 2, title: "Prepare REST exercise", completed: true }  
]
```

## Reading a Collection: GET /tasks

- GET /tasks targets the task **collection**
- The handler sends the current array of tasks
- res.status(200).json() serializes the array as **JSON**
- The response represents the current collection state

```
app.get("/tasks", (req, res) => {  
  res.status(200).json(tasks)  
})
```

## Item Paths: `/tasks/:id`

- An item endpoint needs a way to identify one **resource**
- Express route definitions can contain **path parameters**
- `:id` means “match a value here and store it as `id`”
- The client sends a real path such as `/tasks/1`

| Express Route Pattern   | Real Request Path      | Extracted Value                                       |
|-------------------------|------------------------|-------------------------------------------------------|
| <code>/tasks/:id</code> | <code>/tasks/1</code>  | <code>req.params.id</code> $\equiv$ <code>"1"</code>  |
| <code>/tasks/:id</code> | <code>/tasks/42</code> | <code>req.params.id</code> $\equiv$ <code>"42"</code> |

## Path Data: `req.params`

- Path parameters identify a specific **resource**
- Express stores matched path values in `req.params`
- Parameter values arrive as **strings**
- Numeric identifiers should be converted before comparison

```
app.get("/tasks/:id", (req, res) => {  
  const taskId = Number(req.params.id)  
})
```

## Reading One Task: GET /tasks/:id

- The route extracts the requested **id**
- The array is searched for a matching task
- Existing tasks are returned as **JSON**
- Missing tasks receive a 404 Not Found response

```
app.get("/tasks/:id", (req, res) => {  
  const taskId = Number(req.params.id)  
  const task = tasks.find(task => task.id === taskId)  
  
  if (!task) {  
    return res.status(404).json({ error: "Task not found" })  
  }  
  
  res.status(200).json(task)  
})
```

# Example: Source Code

- Sample **resource**: `AD_read-tasks/`
- `server.js` stores tasks in an in-memory **array**
- `GET /tasks` returns the task collection
- `GET /tasks/:id` returns one task or a **404** response

```
AD_read-tasks/  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── client.js  
├── server.js  
└── package.json
```

# Example: Server Routes

- Sample **resource**: `AD_read-tasks/server.js`
- The first route returns the whole **collection**
- The second route reads one resource by **id**
- Missing identifiers produce a `404 Not Found` response

```
const tasks = [  
  { id: 1, title: "Review backend slides", completed: false },  
  { id: 2, title: "Prepare REST exercise", completed: true }  
]  
  
app.get("/tasks", (req, res) => {  
  res.status(200).json(tasks)  
})  
  
app.get("/tasks/:id", (req, res) => {  
  const taskId = Number(req.params.id)  
  const task = tasks.find(task => task.id === taskId)  
  if (!task) { return res.status(404).json({ error: "Task not found" }) }  
  res.status(200).json(task)  
})
```

# Example: Static Document

- Sample **resource**: `AD_read-tasks/public/index.html`
- The document contains a list for the task **collection**
- The frontend fills the list after receiving the JSON **array**

```
<main class="page">
  <h1>Tasks</h1>
  <p class="status"></p>
  <ul class="task-list"></ul>
</main>
```

# Example: Client Setup

- Sample **resource**: `AD_read-tasks/public/client.js`
- The script selects the task list and the status **element**
- `checkStatus()` verifies whether the HTTP response is **successful**
- `showError()` displays a readable error **message**

```
const taskList = document.querySelector(".task-list")
const status = document.querySelector(".status")

function checkStatus(response) {
  if (!response.ok) {
    throw new Error("The tasks could not be loaded.")
  }

  return response
}

function showError(error) {
  status.textContent = error.message
}
```

# Example: Rendering Tasks

- `fetch("/tasks")` requests the full **collection**
- `response.json()` parses the JSON array
- `showTasks()` creates one list item for each **task**
- The Promise chain connects request, parsing, rendering, and **error handling**

```
function showTasks(tasks) {  
  taskList.replaceChildren()  
  
  for (const task of tasks) {  
    const item = document.createElement("li")  
    item.textContent = task.title  
    taskList.append(item)  
  }  
}  
  
fetch("/tasks")  
  .then(checkStatus).then(response => response.json()).then(showTasks).catch(showError)
```

# Example: Request Flow

- The browser loads the static **frontend**
- The script sends a `GET /tasks` request to the backend **route**
- Express returns a successful JSON **array**
- The frontend renders one list item for each **task**

```
GET /  
  → public/index.html
```

```
GET /style.css  
  → public/style.css
```

```
GET /client.js  
  → public/client.js
```

```
GET /tasks  
  → 200 OK  
  → [  
    { "id": 1, "title": "Review backend slides", "completed": false },  
    { "id": 2, "title": "Prepare REST exercise", "completed": true }  
  ]
```

# Example: Result

- `GET /tasks` returns a JSON **array** of task resources
- The frontend parses the response with `response.json()`
- JavaScript creates one list item for each **task**
- Reading resources does not change server-side **data**

## Tasks

- Review backend slides
- Prepare REST exercise

## Request Data as API Input

- Backend routes do not only receive a **method** and a **path**
- Requests can also include values that guide the backend behavior
- Express exposes these values through different **request properties**
- Choosing the right location keeps the API easier to read and test

# Where Request Data Lives

- HTTP requests can carry data in different **locations**
- Each location has a different role in the API **contract**
- Path and query values come from the request **URL**
- Body data carries structured information sent by the **client**

| Data Location  | Express Property        | Typical Use                      |
|----------------|-------------------------|----------------------------------|
| Path parameter | <code>req.params</code> | Select one resource              |
| Query string   | <code>req.query</code>  | Refine a collection request      |
| Body           | <code>req.body</code>   | Send resource data after parsing |

## Query Data: `req.query`

- Query strings add optional values after the **path**
- They often refine collection reads through filtering, searching, or sorting
- Express stores query values in `req.query`
- Query values arrive as **strings**, even when they represent numbers or booleans

```
GET /tasks?completed=true  
GET /tasks?search=slides
```

# Filtering a Collection

- A query parameter can refine the returned **collection**
- `completed=true` can request only completed tasks
- The handler should define clear behavior when the query is **missing**
- Query values should be converted before boolean or numeric comparisons

```
app.get("/tasks", (req, res) => {  
  const completed = req.query.completed  
  
  if (completed === "true") {  
    return res.status(200).json(tasks.filter(task => task.completed))  
  }  
  
  res.status(200).json(tasks)  
})
```

# Choosing the Data Location

- Put the resource identity in the **path**
- Put optional refinements in the **query string**
- Put submitted resource data in the **body**
- Put message metadata in **headers**

| Question        | Data Location  | Example                                     |
|-----------------|----------------|---------------------------------------------|
| Which resource? | Path parameter | <code>/tasks/1</code>                       |
| Which subset?   | Query string   | <code>/tasks?completed=true</code>          |
| Which new data? | Body           | <code>{ "title": "New task" }</code>        |
| Which format?   | Header         | <code>Content-Type: application/json</code> |

# HTTP Request Parts

- A `POST` request contains more than the target **endpoint**
- The **method** and **path** describe the operation and target resource
- **Headers** describe how the message should be interpreted
- The **body** carries the submitted resource data

| Part   | Example                                           | Role              |
|--------|---------------------------------------------------|-------------------|
| Method | <code>POST</code>                                 | Operation         |
| Path   | <code>/tasks</code>                               | Target collection |
| Header | <code>Content-Type: application/json</code>       | Body format       |
| Body   | <code>{ "title": "Prepare REST exercise" }</code> | Resource data     |

# Parsed Body Data

- The request body is separate from the request **URL**
- Express needs to know how the body should be **parsed**
- `express.json()` parses JSON bodies before route handlers run
- Parsed JSON values become available through `req.body`

```
app.use(express.json())

app.post("/tasks", (req, res) => {
  const submittedTitle = req.body.title
})
```

# Headers in Express

- Request headers are sent by the **client** and can be read through `req`
- Response headers are sent by the **server** through `res`
- Some headers are set **automatically** by methods such as `res.json()`
- Other headers can be set through specific methods or manually with `res.set()`

```
res
  .status(201)
  .location(`/tasks/${task.id}`)
  .json(task)
```

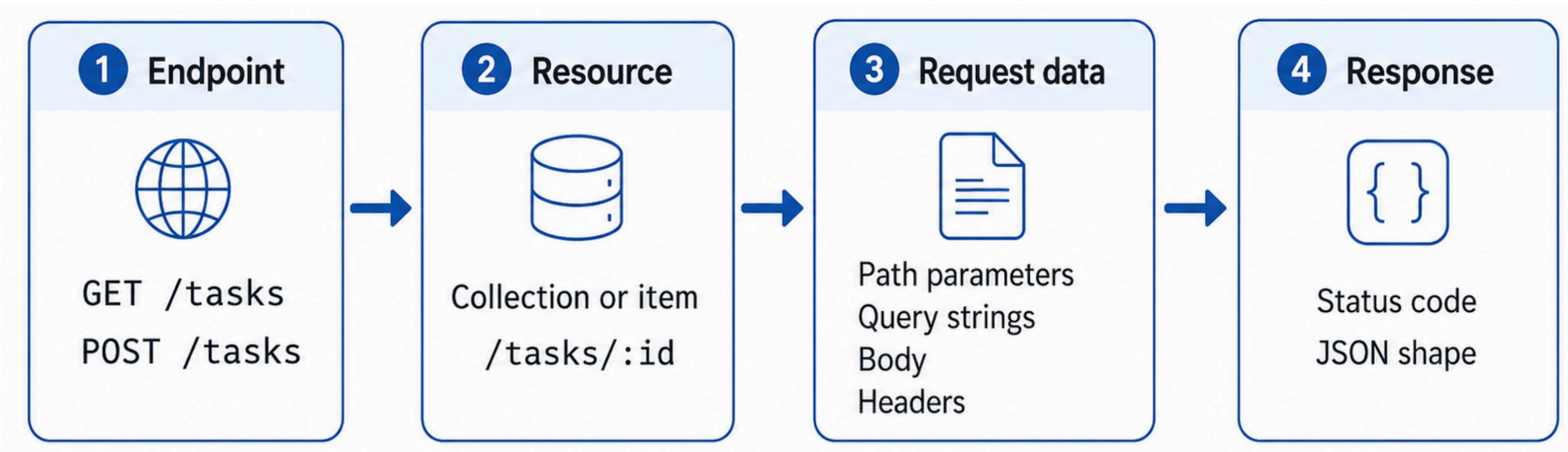
```
res
  .status(201)
  .set("Location", `/tasks/${task.id}`)
  .json(task)
```

# Testing API Requests

- API endpoints can be tested directly before connecting the full **frontend**
- A direct request checks whether the backend route returns the expected **status code** and data
- Tools such as **Postman** and **cURL** can send requests with methods, headers, and bodies
- Testing the API first helps separate backend problems from frontend **rendering** problems

# Reading an Existing API

- A documented API can be explored before looking at its **implementation**
- Endpoint names reveal which **resources** the application exposes
- Request examples show where data is sent and what response is expected



## Reading and Changing State

- `GET` requests retrieve resource **representations**
- They should not change server-side **state**
- `POST`, `PUT`, `PATCH`, and `DELETE` ask the backend to change resources
- State-changing requests require stricter checks on submitted **data**

## Collection Creation: `POST /tasks`

- `POST /tasks` asks the backend to create a task inside the **collection**
- The client sends the task data, but does not choose the final **identifier**
- The server combines submitted data with server-generated **values**
- A successful creation returns `201 Created` and the created representation

# Submitted Data and Created Resource

- The request body contains the values submitted by the **client**
- The server adds values such as the **id** and default state
- The stored resource follows the API **representation**
- The response can return the complete created **task**

```
{  
  "title": "Prepare REST exercise"  
}
```

-----  
Request body

```
{  
  "id": 3,  
  "title": "Prepare REST exercise",  
  "completed": false  
}
```

-----  
Created resource

# JSON Requests with `fetch()`

- `fetch()` receives request options as a JavaScript **object**
- `method: "POST"` selects the creation operation
- `Content-Type` tells the backend that the body contains **JSON**
- `JSON.stringify()` converts the JavaScript object into a JSON **string**

```
fetch("/tasks", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({ title: "Prepare REST exercise" })  
})
```

## Submitted Title: `req.body.title`

- `POST /tasks` reads the submitted title from the parsed **body**
- The backend should not use the value before checking it
- The submitted title becomes part of the new **resource**
- Server-generated values complete the final representation

```
app.post("/tasks", (req, res) => {  
  const submittedTitle = req.body.title  
})
```

# Resource Shape After Creation

- The created resource should have the same shape as other **tasks**
- Client data is combined with server-generated values
- The server-side collection stores the complete **resource**
- The response can return the final representation to the client

```
const task = {  
  id: nextTaskId,  
  title,  
  completed: false  
}
```

# Example: Source Code

- Sample **resource**: `AE_create-task/`
- `server.js` exposes `GET /tasks` and `POST /tasks`
- `client.js` sends a **JSON body** with `fetch()`
- The created task is added to the visible **list**

```
AE_create-task/  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── client.js  
├── server.js  
└── package.json
```

# Example: Server Route

- Sample **resource**: `AE_create-task/server.js`
- `POST /tasks` reads, normalizes, and validates the submitted **title**
- Invalid client data returns `400 Bad Request`
- A valid request creates the task and returns `201 Created`

```
app.use(express.json())

let nextTaskId = 3

app.post("/tasks", (req, res) => {
  const submittedTitle = req.body.title

  if (typeof submittedTitle !== "string") {
    return res.status(400).json({ error: "Title is required" })
  }

  const title = submittedTitle.trim()

  if (!title) {
    return res.status(400).json({ error: "Title is required" })
  }

  const task = { id: nextTaskId, title, completed: false }
  nextTaskId += 1
  tasks.push(task)

  res.status(201).json(task)
})
```

# Example: Static Document

- Sample **resource**: `AE_create-task/public/index.html`
- The form collects the new task **title**
- The list shows the current task collection
- JavaScript controls the submission with an event listener

```
<main class="page">
  <h1>Tasks</h1>

  <form class="task-form">
    <label for="task-title">New task</label>
    <input id="task-title" name="title" required>
    <button type="submit">Add task</button>
  </form>

  <p class="status"></p>
  <ul class="task-list"></ul>
</main>
```

# Example: Submit Handler

- The request starts when the user submits the **form**
- JavaScript intercepts the submit event
- `event.preventDefault()` stops normal form navigation
- The handler calls `createTask()` to send the **POST** request

```
function handleSubmit(event) {  
  event.preventDefault()  
  
  createTask(titleInput.value)  
    .then(readJsonOrThrow)  
    .then(handleCreatedTask)  
    .catch(showError)  
}  
  
taskForm.addEventListener("submit", handleSubmit)
```

# Example: Client Request

- `createTask()` sends the HTTP **POST** request
- The request targets the task **collection**
- The header declares that the body contains **JSON**
- The function returns the pending response as a **Promise**

```
function createTask(title) {  
  return fetch("/tasks", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({ title })  
  })  
}
```

# Example: Handling the Response

- The response body is parsed as **JSON**
- Failed responses become errors in the Promise chain
- Successful responses provide the created **task**
- The server message can be shown in the interface

```
function readJsonOrThrow(response) {  
  return response.json().then(data => {  
    if (!response.ok) {  
      throw new Error(data.error || "Request failed")  
    }  
  
    return data  
  })  
}
```

## Example: Updating the Interface

- A successful response adds the created task to the **list**
- The input is cleared after the task is added
- A failed request shows a readable **status message**
- The frontend reacts to the outcome chosen by the backend

```
function handleCreatedTask(task) {  
  appendTask(task)  
  titleInput.value = ""  
  status.textContent = ""  
}  
  
function showError(error) {  
  status.textContent = error.message  
}
```

# Example: Request Flow

- The form submission sends a `POST /tasks` request
- The request body carries the submitted **title**
- The backend validates the body and creates the **task**
- The response returns the created resource as **JSON**

```
POST /tasks
Content-Type: application/json

{ "title": "Prepare REST exercise" }

→ 201 Created
→ {
  "id": 3,
  "title": "Prepare REST exercise",
  "completed": false
}
```

# Example: Result

- The form sends a JSON body to `POST /tasks`
- The backend validates the submitted **title**
- The server creates a task and returns `201 Created`
- The frontend appends the returned resource to the **list**

## Tasks

Add task

- Review backend slides
- Prepare REST exercise

## Trust Boundaries in APIs

- Client data should never be trusted as already **valid**
- Frontend checks improve the user experience, but they can be bypassed
- Requests can come from forms, scripts, API clients, or manual tools
- Server-side validation protects the consistency of the application **state**

# Server-Side Validation

- Validation runs inside the backend **route**
- The handler checks submitted data before changing server-side **state**
- Invalid requests stop the route with an early **response**
- Valid requests continue toward resource creation or update

# Validation Flow

- Read the **submitted values** from `req.body`
- Normalize values before checking them
- Check required fields, expected types, and allowed ranges
- Stop before changing the collection when data is invalid
- Return a clear `400 Bad Request` **response**

# Validating Submitted Values

- The title is expected to be a non-empty **string**
- String values should be trimmed before checking emptiness
- Invalid data stops the creation flow before the collection changes
- A validation error returns `400 Bad Request`

```
const submittedTitle = req.body.title

if (typeof submittedTitle !== "string") {
  return res.status(400).json({ error: "Title is required" })
}

const title = submittedTitle.trim()

if (!title) {
  return res.status(400).json({ error: "Title is required" })
}
```

# Consistent Error Responses

- Error responses should follow a predictable **shape**
- The frontend can read the same field across different failed requests
- A short message is enough for simple APIs
- More complex APIs may add details for specific **fields**

```
{  
  "error": "Title is required"  
}
```

```
{  
  "error": "Task not found"  
}
```

# Creation Outcomes

- A successful creation returns `201 Created`
- Invalid client data returns `400 Bad Request`
- The frontend should branch on the actual **status code**
- The response body provides either the created resource or an **error message**

| Situation        | Status Code                  | Frontend Reaction        |
|------------------|------------------------------|--------------------------|
| Task created     | <code>201 Created</code>     | Add the task to the list |
| Missing title    | <code>400 Bad Request</code> | Show the error message   |
| Empty title      | <code>400 Bad Request</code> | Keep the form editable   |
| Wrong title type | <code>400 Bad Request</code> | Show the error message   |

# Other Resource Changes

- REST APIs can also change or remove existing **items**
- State-changing operations usually target an item path such as `/tasks/:id`
- Each method communicates a different kind of **change**
- The backend should validate submitted data before changing server-side **state**

| Method | Endpoint                | Main Idea              |
|--------|-------------------------|------------------------|
| PUT    | <code>/tasks/:id</code> | Replace one resource   |
| PATCH  | <code>/tasks/:id</code> | Change selected fields |
| DELETE | <code>/tasks/:id</code> | Remove one resource    |

# Replacing Resources: PUT

- PUT usually sends a complete new **representation** for the resource
- The request targets one existing **item**
- The submitted representation replaces the current resource state
- Missing resources usually return 404 Not Found

```
PUT /tasks/1
Content-Type: application/json

{
  "title": "Prepare REST exercise",
  "completed": true
}
```

# Implementing Partial Updates: PATCH

- PATCH changes selected fields of an existing **item**
- The item path identifies which resource should change
- The body contains only the submitted **changes**
- Submitted values should be validated before updating the resource

```
PATCH /tasks/1
Content-Type: application/json

{
  "completed": true
}
```

# Updating Resources: PATCH

- The route first searches for the requested **task**
- Missing resources return 404 Not Found
- Invalid submitted values return 400 Bad Request
- A successful update returns the updated **representation**

```
app.patch("/tasks/:id", (req, res) => {
  const taskId = Number(req.params.id)
  const task = tasks.find(task => task.id === taskId)

  if (!task) {
    return res.status(404).json({ error: "Task not found" })
  }

  if (typeof req.body.completed !== "boolean") {
    return res.status(400).json({ error: "Completed must be a boolean" })
  }

  task.completed = req.body.completed
  res.status(200).json(task)
})
```

# Deleting Resources: DELETE

- DELETE removes an existing **item**
- The item path identifies which resource should be removed
- Missing resources return 404 Not Found
- Successful deletion often returns 204 No Content

```
app.delete("/tasks/:id", (req, res) => {  
  const taskId = Number(req.params.id)  
  const taskIndex = tasks.findIndex(task => task.id === taskId)  
  
  if (taskIndex === -1) {  
    return res.status(404).json({ error: "Task not found" })  
  }  
  
  tasks.splice(taskIndex, 1)  
  res.status(204).end()  
})
```

# Summary

- REST organizes backend APIs around **resources**, collections, and items
- `GET` retrieves resource **representations** without changing server-side state
- Path parameters and query strings identify or refine **read requests**
- `POST` creates new resources inside a **collection**
- `PUT` and `PATCH` change existing resources in different ways
- Request bodies carry structured **client data**
- Server-side validation protects API state from invalid **requests**
- Status codes describe the outcome of each API **operation**

# Bibliography

- Holmes, E., & Harms, D. (2016). *Express in Action*. Manning Publications
  - Chapters 1-3: Introducing Express, Routing, and Responses
- Fielding, R. T. (2000). *Architectural Styles of Network-based Software Architectures*
  - <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- MDN Web Docs. *HTTP request methods*
  - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>
- MDN Web Docs. *HTTP response status codes*
  - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>
- MDN Web Docs. *JSON*
  - [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/JSON](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON)

# Bibliography (cont.)

- Express.js Official Docs. *Basic routing*
  - <https://expressjs.com/en/starter/basic-routing.html>
- Express.js Official Docs. *API Reference: Request*
  - <https://expressjs.com/en/api.html#req>
- Express.js Official Docs. *API Reference: Response*
  - <https://expressjs.com/en/api.html#res>
- Express.js Official Docs. *express.json()*
  - <https://expressjs.com/en/api.html#express.json>
- Postman Learning Center. *Document your APIs*
  - <https://learning.postman.com/docs/publishing-your-api/api-documentation-overview>
- Readersourcing 2.0. *API Documentation*
  - <https://documenter.getpostman.com/view/4632696/RWTiwfV4>